

Enterprise Cloud Incident Response and Forensic Readiness on AWS

Author: Adedayo

Specialization: Cloud Security & Incident Response

Platform: Amazon Web Services (AWS)

1. Problem Statement.

A mid-sized financial institution lacked a formal cloud incident response process. Security events were detected but lacked structured investigation and containment workflows. The organization required a cloud-specific incident response and forensic readiness framework.

implemented this framework to ensure that security incidents involving IAM credentials, malicious API activity, data exposure, and infrastructure misconfigurations can be detected, investigated, contained, and remediated efficiently.

The framework integrates AWS-native security services, documented response playbooks, forensic preservation procedures, and automated remediation workflows.

2. Objectives

The primary objectives were to:

- Improve cloud incident detection and response capability
- Establish standardized response playbooks
- Preserve forensic evidence securely
- Reduce incident response time
- Enable automated containment
- Support audit and compliance requirements

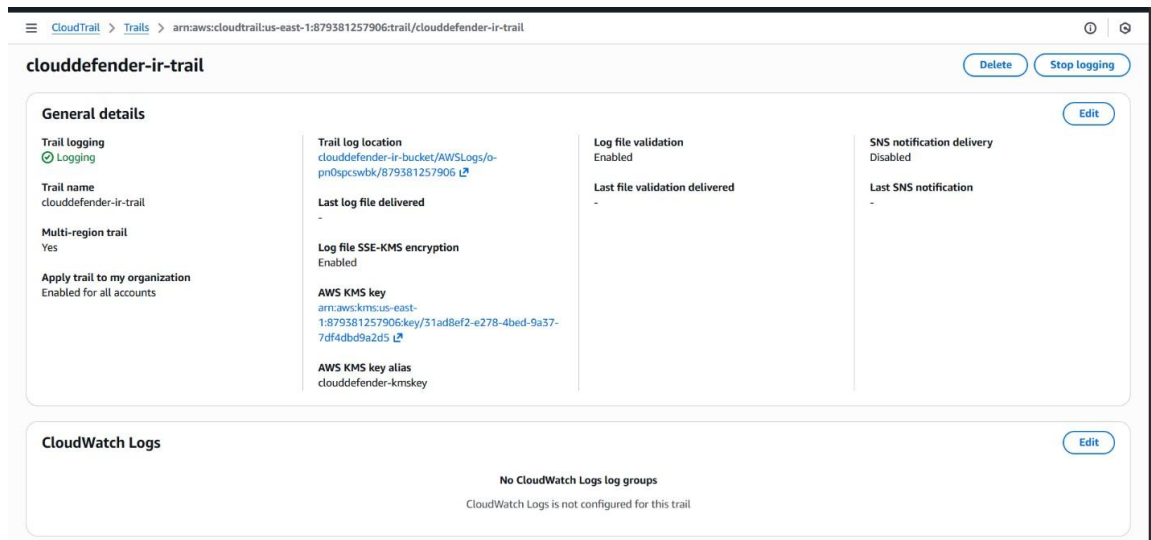
3. Incident Response Evidence Repository

A dedicated S3 bucket was created to store forensic evidence.

The bucket was configured with:

- Encryption enabled

- Versioning enabled
- Restricted access policies
- Deletion protection



This repository stores CloudTrail logs, CloudWatch exports, snapshots, and investigation artifacts.

4. Centralized Logging for Incident Response

AWS CloudTrail was configured for centralized API logging.

CloudTrail serves as the primary forensic data source and supports investigation of:

- Credential compromise
- Privilege escalation
- Resource tampering
- Data access

Logs are automatically delivered to the evidence repository.

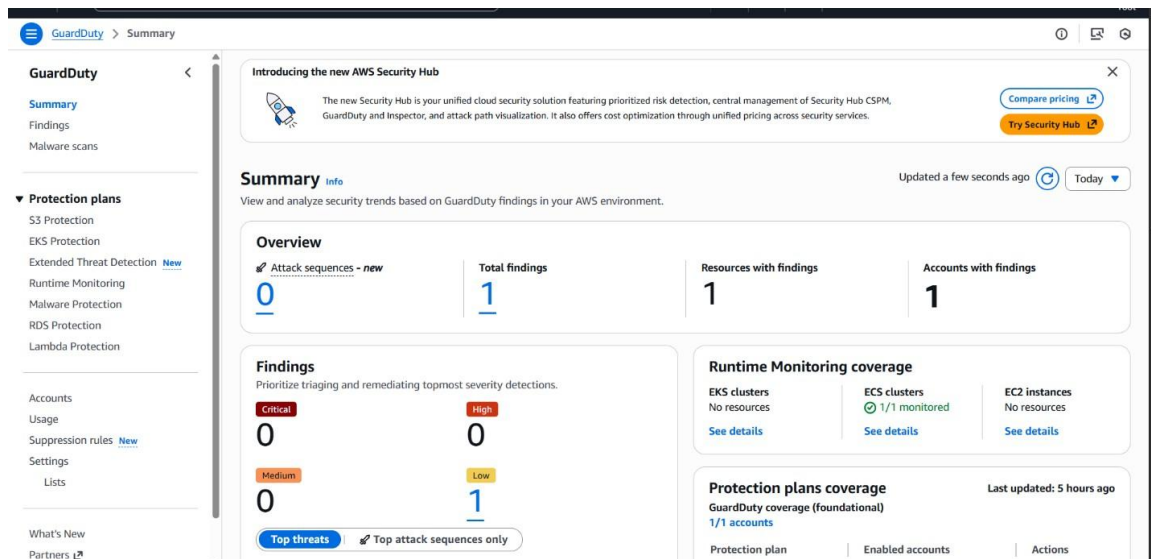
5. Threat Detection Services

5.1 Amazon GuardDuty

GuardDuty was enabled for continuous threat detection.

It supports detection of:

- IAM credential compromise
- Malicious API behavior
- Data exfiltration
- EC2 compromise
- Lambda abuse



GuardDuty findings trigger incident response workflows.

5.2 AWS Security Hub

Security Hub was enabled for centralized findings management.

Findings from GuardDuty and AWS security standards are aggregated and prioritized for response.

6. Incident Simulation and Testing

6.1 Incident 1: IAM Credential Misuse

Detection Source: CloudTrail

CloudTrail > Event history

Event history (7) Info

Event history shows you the last 90 days of management events.

Lookup attributes

User name: test-user

Filter by date and time

Clear filter

Event name	Event time	User name	Event source	Resource type	Resource name
ListUsers	January 18, 2026, 21:42:55 (UTC...)	test-user	iam.amazonaws.com	-	-
ListBuckets	January 18, 2026, 21:42:36 (UTC...)	test-user	s3.amazonaws.com	-	-
DescribeInstances	January 18, 2026, 21:42:25 (UTC...)	test-user	ec2.amazonaws.com	-	-
ListRoles	January 18, 2026, 21:40:46 (UTC...)	test-user	iam.amazonaws.com	-	-
GetCallerIdentity	January 18, 2026, 21:40:19 (UTC...)	test-user	sts.amazonaws.com	-	-
ListUsers	January 18, 2026, 21:38:21 (UTC...)	test-user	iam.amazonaws.com	-	-
GetCallerIdentity	January 18, 2026, 20:49:02 (UTC...)	test-user	sts.amazonaws.com	-	-

Unusual API enumeration activity was detected from an IAM user.

Triage Actions:

- Reviewed CloudTrail events - Identified access key and source IP - Confirmed suspicious behavior.

CloudTrail > Event history > ListUsers

JSON view

```

1 {
2   "eventVersion": "1.11",
3   "userIdentity": {
4     "type": "IAMUser",
5     "principalId": "AIDAAZPZVDZIX338C3M",
6     "arn": "arn:aws:iam::879381257906:user/test-user",
7     "accountId": "879381257906",
8     "accessKeyId": "AKIA4ZPZVDZH34YEPC7",
9     "userName": "test-user"
10  },
11  "eventTime": "2026-01-18T21:42:55Z",
12  "eventSource": "iam.amazonaws.com",
13  "eventName": "ListUsers",
14  "awsRegion": "us-east-1",
15  "sourceIPAddress": "13.222.217.203",
16  "userAgent": "aws-cli/2.32.34 md/awscrt#0.29.1 ua/2.1 os/linuxx86_1.158-180.294.amzn2023.x86_64 md/archx86_64 lang/python#3.13.11 md/pyimp1acPython",
17  "requestParameters": null,
18  "responseElements": null,
19  "requestID": "2adaf339-b8a4-4fd0-9825-da29ff2a7b2f",
20  "eventID": "7a7ad7d8-33fe-4a78-8363-4d0985964e5d",
21  "readOnly": true,
22  "eventType": "AwsApiCall",
23  "managementEvent": true,
24  "recipientAccountId": "879381257906",
25  "eventCategory": "Management",
26  "tlsDetails": {
27    "tlsVersion": "TLSv1.3",
28    "cipherSuite": "TLS_AES_128_GCM_SHA256",
29    "clientProvidedHostHeader": "iam.amazonaws.com"
30  }

```

Severity: SEV 2 (High)

Forensic Preservation:

- CloudTrail logs archived
- IAM metadata documented

Containment:

- Access key deactivated

Outcome:

- No data loss
- Early containment

Lessons Learned:

- Manual log review remains critical
- Early isolation reduces impact

6.2 Incident 2: EC2 Public SSH Exposure

Detection Source: Security Hub

An EC2 instance was found with SSH open to 0.0.0.0/0.

Triage Actions:

- Reviewed security group rules
- Verified public IP exposure

Severity: SEV 2 (High)

Forensic Preservation:

- EBS snapshot created
- Metadata recorded

Containment:

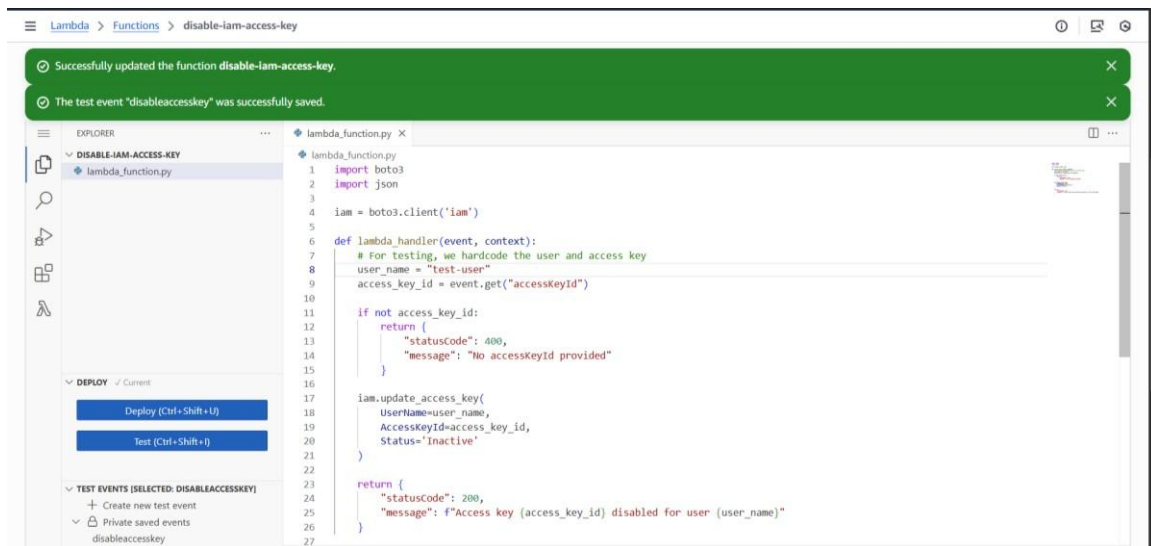
- SSH access restricted

Outcome:

- No unauthorized access
- Risk eliminated

7. Automated Remediation Framework

An automated remediation workflow was implemented using AWS Lambda.



The workflow:

- Receives compromised access key IDs
- Disables keys
- Logs actions to CloudWatch

Keys are deactivated, not deleted, to preserve evidence.

Testing confirmed successful automated containment.

Lambda > Functions > disable-iam-access-key

Successfully updated the function disable-iam-access-key.

The test event "disableaccesskey" was successfully saved.

Executing function: succeeded (logs)

Details

```
{
  "statusCode": 200,
  "message": "Access key AKIA4ZPZVDZ34YEPC7 disabled for user test-user"
}
```

Summary

Code SHA-256

R6fL2f5y9xHPT04N0jRtY2hz04DIAxWgwXm4xSMnwRQ=

Function version

\$LATEST

Duration

295.03 ms

Resources configured

128 MB

Init duration

501.38 ms

Log output

Execution time

31 seconds ago

Request ID

8fe9c674-9e5a-4a9a-bbad-6b79ec7c193b

Billed duration

797 ms

Max memory used

91 MB

IAM > Users > test-user

test-user Info

Delete

Identity and Access Management (IAM)

Search IAM

Dashboard

Access Management

User groups

Users

Roles

Policies

Identity providers

Account settings

Root access management

Temporary delegation requests

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Credential report

Organization activity

Service control policies

Summary

ARN

arn:aws:iam::879381257906:user/test-user

Console access

Disabled

Created

January 18, 2026, 19:31 (UTC)

Last console sign-in

-

Access key 1

AKIA4ZPZVDZ34YEPC7 - Inactive

Used Yesterday. Created yesterday.

Access key 2

Create access key

Permissions

Groups

Tags

Security credentials

Last Accessed

Permissions policies (1)

Remove

Add permissions

Permissions are defined by policies attached to the user directly or through groups.

Search

Filter by Type

All types

Policy name

Type

Attached via

ReadOnlyAccess

AWS managed - job function

Directly

Permissions boundary (not set)

Generate policy based on CloudTrail events

CloudWatch > Log management > /aws/lambda/disable-iam-access-key > 2026/01/19/[\$LATEST]0b641ecca99540629cfc7d6631fcdfb

CloudWatch

Favorites and recents

Ingestion

Dashboards

Alarms

AI Operations

GenAI Observability

Application Signals (APM)

Infrastructure Monitoring

Logs

Log Management

Log Anomalies

Live Tail

Logs Insights

Contributor Insights

Log events

Actions

Start tailing

Create metric filter

You can use the filter bar below to search for and match terms, phrases, or values in your log events.

Filter events - press enter to search

Clear

1m

30m

1h

12h

Custom

UTC timezone

Display

Timestamp

Message

No older events at this moment. Retry

2026-01-19T22:52:43.211Z

INIT_START Runtime Version: python:3.14.v32 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime::ee4e6d61a50fb29d03b87572cc6270ba..

2026-01-19T22:52:43.716Z

START RequestId: 8fe9c674-9e5a-4a9a-bbad-6b79ec7c193b Version: \$LATEST

2026-01-19T22:52:44.012Z

END RequestId: 8fe9c674-9e5a-4a9a-bbad-6b79ec7c193b

2026-01-19T22:52:44.012Z

REPORT RequestId: 8fe9c674-9e5a-4a9a-bbad-6b79ec7c193b Duration: 295.03 ms Billed Duration: 797 ms Memory Size: 128 MB Max Memory Use..

No newer events at this moment. Auto retry paused. Resume

8. Alerting for Logging Tampering

CloudWatch alarms were created to detect:

- StopLogging
- DeleteTrail

These alerts provide early warning of audit log tampering.

9. Incident Response Playbooks

Cloud-specific response playbooks were developed for:

- Credential compromise
- API abuse
- Data exfiltration
- Resource misconfiguration

Each playbook defines detection, triage, preservation, containment, and escalation steps.

10. Forensic Preservation Procedures

Tested procedures include:

- CloudTrail archival
- CloudWatch export to S3
- EBS snapshots
- IAM key deactivation

All evidence is encrypted and access-controlled.

11. Governance and Documentation

All response workflows, evidence handling procedures, and automation processes were documented.

Documentation supports audits, investigations, and team readiness.

12. Outcomes and Impact

This implementation delivered:

- Faster incident response
- Strong forensic readiness
- Automated containment
- Improved detection coverage
- Reduced operational risk
- Audit-ready processes

13. Conclusion

I designed and implemented a comprehensive cloud incident response and forensic readiness framework on AWS.

Through integrated detection, evidence preservation, automation, and governance, this solution enables effective and resilient security operations.